

Linear System Identification for an Inverted Pendulum Using Convex Optimization

EID 488: Convex Optimization Techniques

Author:

KHUSHANT KHURANA

Professor:

PROF. DAVIS

May 12, 2025

CONTENTS

I	Abstract	2
II	Introduction	2
III	Problem Description	2
III-A	Inverted Pendulum System	3
III-B	State Space Paradigm	3
III-C	Designing Linear Quadratic Controller for Unstable Equilibrium	3
III-D	Enforcing Stability Constraints Through Lyapunov's Stability Criteria	4
III-E	Physical Constraints on Decision Variables	4
IV	Data Acquisition Using Gazebo	4
V	Problem Formulation For Stable Equilibrium using casADi	4
V-A	Proving the Solution's Existence	5
V-B	casADi's Solver Specifics	5
VI	Problem Formulation For Unstable Equilibrium Using casADi	5
VI-A	Proving the Solution's Existence	6
VII	Results	6
VII-A	Stable Equilibrium Optimization Results	6
VII-B	Unstable Equilibrium Optimization Results	8
VIII	Conclusion	10
IX	Appendix I. Optimization Script for Stable Equilibrium	11
X	Appendix II. Optimization Script for Unstable Equilibrium	12

Linear System Identification for an Inverted Pendulum Using Convex Optimization

EID 488: Convex Optimization Techniques

I. ABSTRACT

Linear system models are widely regarded as effective due to their scalability to arbitrary inputs, interpretability, and computational efficiency. Consequently, significant research has focused on developing algorithms to approximate nonlinear systems with linear models that capture essential dynamics. In this work, a similar approach is applied to identify linear system representations of an inverted pendulum—a canonical benchmark problem—around its stable and unstable equilibrium points.

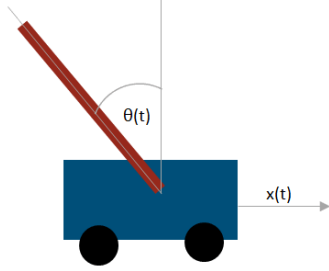


Fig. I.1: Inverted pendulum system

High-fidelity simulation data of the real-world pendulum is obtained from a nonlinear physics simulator. The system identification task is formulated as a constrained optimization problem that minimizes the least-squares error between the true and estimated states. The optimization is implemented using the CasADi framework in Python. For the stable and unstable equilibrium conditions, the optimal cost function values are 0.08 % and 0.20 %, respectively. The resulting state-space matrices are presented in the relevant sections.

Model validation is performed by comparing the pendulum's oscillatory behavior as predicted by both the nonlinear and linear models. The linear system corresponding to the stable equilibrium captures the oscillations with high fidelity, whereas the model for the unstable equilibrium fails to do so.

II. INTRODUCTION

System identification is the process of building mathematical models of dynamic systems using observed data [1]. Instead of deriving models purely from physical laws (such as Newton's laws), system identification uses measurements of a system's input and output to infer its underlying behavior. This can be done in two ways: *static system identification*, which involves fitting the output y using simple actuation functions

$h(u)$ and *dynamic system identification*, which seeks to learn the state-space representation of the system. In dynamic system identification, the goal is to find differential or difference equations such as those shown in Equation 1 that model the system's evolution over time [2].

$$\begin{aligned}\dot{x} &= Ax + Bu & (\text{state evolution}) \\ y &= Cx + Du & (\text{output equation})\end{aligned}\tag{1}$$

In this formulation:

- x is the **state vector**, representing the internal configuration of the system (example: position and velocity);
- u is the **input vector**, representing external controls or excitations (example: motor power);
- y is the **output vector**, representing observable measurements (example: sensor measurements);
- A is the **system dynamics matrix**, encoding how the state evolves on its own (stability, natural response);
- B is the **input matrix**, defining how inputs affect the state;
- C is the **output matrix**, specifying how internal states contribute to measurements;
- D is the **feedthrough matrix**, describing how the input directly affects the output.

Performing system identification for nonlinear systems offers valuable insight into the underlying dynamic behavior. For instance, linear state-space models provide interpretable structures that enable the estimation of key system properties, such as poles (which indicate system stability), damping ratios (which characterize time-domain response), and other dynamic characteristics [3].

Convex optimization has attracted growing interest for system identification due to its effectiveness in sparse and constrained problems, with successful applications in signal processing, statistics, and machine learning.[4] This paper demonstrates its use in identifying a linear model of an inverted pendulum for both of its equilibrium positions.

III. PROBLEM DESCRIPTION

An inverted pendulum possesses two equilibrium points: a stable equilibrium when the pendulum is vertically downward, and an unstable equilibrium when it is vertically upward. In this study, system identification is carried out for both configurations. The unstable equilibrium, in particular, necessitates continuous control input to maintain balance, which introduces additional complexity to the optimization problem.

The following formulation outlines the general procedure applicable to both cases, while the specific problem setup and results for each equilibrium point are detailed in subsequent sections.

A. Inverted Pendulum System

The "inverted pendulum" case study is a coupled dynamical system consisting of a pendulum pivoted about a cart's center of mass. In order to move the pendulum, a force is applied to the cart which provides the necessary torque for the pendulum to swing. A schematic of the system in study is shown in Figure III.1.

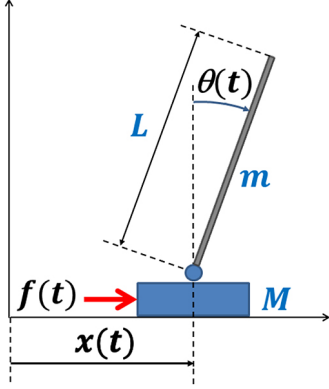


Fig. III.1: Schematic of the inverted pendulum system [5]

The nonlinear model for the system is given in Equation 2 where m and M are mass of the pendulum and the cart respectively, I is pendulum's inertia through its center of mass, l is pendulum's half length, θ is pendulum's angle from vertical, x is the cart's displacement, b is pendulum's damping, and F is the force applied to the cart.

$$\begin{aligned} (M + m)\ddot{x} + b\dot{x} + ml\ddot{\theta}\cos(\theta) - ml\dot{\theta}^2\sin(\theta) &= F \\ (I + ml^2)\ddot{\theta} + mgl\sin(\theta) &= -ml\ddot{x}\cos(\theta) \end{aligned} \quad (2)$$

The time derivatives of θ and x are strongly coupled due to the bidirectional interaction: forces applied to the cart affect the pendulum's motion, while the pendulum's swing, in turn, imparts reactive forces on the cart.

B. State Space Paradigm

The objective of this study is to determine a linear system representation of an inverted pendulum. Linear systems can be formulated in the state space representation given in Equation 1. Accordingly, an analogous representation can be generated for an inverted pendulum. The corresponding state vector $\vec{x}$ in $\mathbb{R}^4$ is given in Equation 3 and the full state space formulation with appropriate A, B, C, D matrices is given in Equation 4.

$$\vec{x} = \begin{bmatrix} \text{cart position } (x) \\ \text{pendulum angle } (\theta) \\ \text{cart velocity } (\dot{x}) \\ \text{pendulum angular velocity } (\dot{\theta}) \end{bmatrix} \quad (3)$$

$$\dot{\vec{x}} = \begin{bmatrix} \dot{x} \\ \dot{\theta} \\ \ddot{x} \\ \ddot{\theta} \end{bmatrix} = \begin{bmatrix} a_{11} & a_{12} & a_{13} & a_{14} \\ a_{21} & a_{22} & a_{23} & a_{24} \\ a_{31} & a_{32} & a_{33} & a_{34} \\ a_{41} & a_{42} & a_{43} & a_{44} \end{bmatrix} \vec{x} + \begin{bmatrix} b_{11} \\ b_{21} \\ b_{31} \\ b_{41} \end{bmatrix} F \quad (4)$$

$$y = \begin{bmatrix} c_{11} & c_{12} & c_{13} & c_{14} \\ c_{21} & c_{22} & c_{23} & c_{24} \\ c_{31} & c_{32} & c_{33} & c_{34} \\ c_{41} & c_{42} & c_{43} & c_{44} \end{bmatrix} \vec{x} + \begin{bmatrix} d_{11} \\ d_{21} \\ d_{31} \\ d_{41} \end{bmatrix} F$$

For simplicity, the output matrix C is assumed to be the identity matrix, implying that all elements of the state vector are directly measurable. Additionally, the feedthrough matrix D is set to zero, indicating that the control input does not directly influence the measured output. Under these assumptions, the system identification problem reduces to estimating the entries of the state transition matrix A and the input matrix B , resulting in a total of 20 decision variables.

To minimize the error between true states X_{True} and estimated states $X_{\text{Estimated}}$, a least-squares objective is formulated as defined in Equation 5. The estimated states are computed using the state-space formulation described in Equation 4. Here, N denotes the number of time steps in the dataset obtained from the Gazebo physics simulator [6]. The weighting matrix Q is a scaled identity matrix, used to assign relative importance to each state variable in the cost function. Since Q is positive definite (i.e., all eigenvalues are strictly greater than zero), the resulting optimization problem is strictly convex, ensuring a unique global minimum.

$$\text{Cost}(J) = \sum_{i=1}^{N-1} (X_{\text{True}} - X_{\text{Estimated}})^T Q (X_{\text{True}} - X_{\text{Estimated}}) \quad (5)$$

C. Designing Linear Quadratic Controller for Unstable Equilibrium

As explained before, the unstable equilibrium for the pendulum can only be achieved with a constant control input. Therefore, a controller needs to be specified in the optimization problem for the solver to find the desired answer. In this study, the control input F is governed by a Linear Quadratic Controller (LQR). This is designed in Robot Operating System (ROS2) [7] which is then integrated with the Gazebo physics simulator [8] to control the pendulum. The gains of the LQR along with the calculation of f is given in Equation 6. Note, these gains of the LQR are determined using iterative tuning.

$$\begin{aligned} K_{\text{LQR}} &= [0, 1.4, 0, 0.4] \\ F &= -Kx \end{aligned} \quad (6)$$

The advantage of using LQR comes from its ease of ensuring stability. In state space form, closed-loop dynamics (that is, passive dynamics combined with LQR control input) is given in Equation 7.

$$\dot{x} = Ax + B(-K_{\text{LQR}}x) \quad (\text{Closed Loop Dynamics}) \quad (7)$$

The closed-loop dynamics matrix is given by $A_{\text{closed_loop}} = A - BK$, which is another 4×4 matrix. Stability can be ensured by verifying that the real parts of all its eigenvalues lie in the left-half complex plane (i.e., have real parts less than zero). In this problem, this condition is instead imposed as a constraint on the A and B matrices. The calculation of these eigenvalues requires constraints to be non-linear, which also makes this problem NLP (Nonlinear Program).

D. Enforcing Stability Constraints Through Lyapunov's Stability Criteria

When performing system identification at the unstable equilibrium point, the resulting closed-loop system matrix $A_{\text{closed_loop}}$ must be negative definite to ensure system stability. To enforce this requirement, Lyapunov's stability criterion is employed. According to this criterion, a linear system is asymptotically stable if there exists a symmetric positive definite matrix P such that the Lyapunov inequality in Equation 8 is satisfied [9]. Intuitively, this can be interpreted in terms of the system's energy function, $V(x) = x^T Q x$, which must decrease over time and eventually converge to zero. This decay of energy is guaranteed when the eigenvalues of A_{CL} lie strictly in the left half of the complex plane.

$$A^T P + P A \prec 0 \quad (8)$$

A valid positive definite matrix P then would be enough to prove the stability of the matrix A . For simpler dynamics with unnatural weighting, a valid candidate for P is the identity matrix. Doing so constrains the symmetric part of the matrix $A_{\text{closed_loop}}$ to have eigenvalues less than 0, as given in Equation 9. This is enough to ensure asymptotic stability.

$$A_{\text{CL}}^T + A_{\text{CL}} = 2A_{\text{CL}} \prec 0 \quad (9)$$

In order to constrain the eigenvalues, now, Sylvester's criterion is used. For a matrix $A \in \mathbb{R}^{n \times n}$, Sylvester's Criterion for negative definiteness states that the leading principal minors of A_{CL} alternate in sign [10]:

$$(-1)^n \det(A_n) > 0, \text{ where } n \in \mathbb{Z}^+$$

where A_k is the k -th leading principal submatrix of A . This condition ensures that the matrix A is negative definite.

E. Physical Constraints on Decision Variables

To prove that the problem has a feasible solution, the domain must be a closed and bounded set. This condition is necessary to ensure that the cost function attains a minimum, as will be explained later. A bounded set can be defined as follows: if $S \subset \mathbb{R}^n$, then S is bounded if there exists a ball of finite radius ϵ such that

$$\|x\| \leq \epsilon \text{ for all } x \in S.$$

The set S is also closed if it includes its boundary. Consequently, physical constraints C are added to the matrices A and B , as shown in Equation 10 and Equation 11, respectively,

ensuring that the set of feasible parameters lies within a ball in $\mathbb{R}^{20}$ and is therefore bounded.

$$A \in \mathbb{R}^{4 \times 4} \text{ such that } -C \leq a_{ij} \leq C \quad \forall i, j \in \{1, 2, 3, 4\} \quad (10)$$

$$B \in \mathbb{R}^{4 \times 1} \text{ such that } -C \leq b_j \leq C \quad \forall j \in \{1, 2, 3, 4\} \quad (11)$$

IV. DATA ACQUISITION USING GAZEBO

Gazebo is an open-source robotics simulator that allows you to test and develop robotic systems in a virtual environment. [8] It is widely used in research and industry because of its realistic physics engine, sensor simulation, and support for robot models. As mentioned earlier, the true states of the inverted pendulum are generated from Gazebo. A video of the LQR controller stabilizing the pendulum can be found in [6]. The static model and the corresponding reference frame are shown in Figure IV.1.

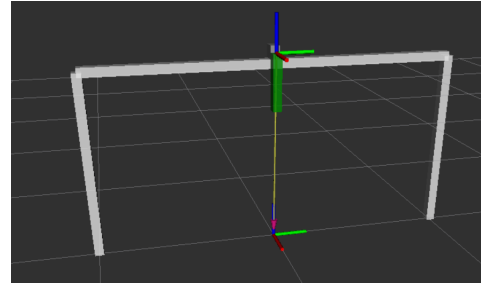


Fig. IV.1: Model of the inverted pendulum and local coordinate frames in Gazebo [6]

The physical parameters for the pendulum and the cart used in the simulation are given in Table I. The sample rate (time step) used for the simulation is 0.002 sec

TABLE I: Physical Parameters of Links in Inverted Pendulum Model

Link	Size (m)	Mass (kg)	Inertia (kg-m ²)
Cart	0.07×0.07×0.05	0.1	$I_{xx} = 0.1$
			$I_{yy} = 0.1$
			$I_{zz} = 0.075$
			$I_{xy} = I_{xz} = I_{yz} = 0$
Pendulum	0.05×0.05×0.3	1.0	$I_{xx} = 0.0104$
			$I_{yy} = 0.0104$
			$I_{zz} = 0.0077$
			$I_{xy} = I_{xz} = I_{yz} = 0$

V. PROBLEM FORMULATION FOR STABLE EQUILIBRIUM USING CASADI

The optimization problem formulated for stable equilibrium is defined below. Since there is no actuation required to keep the system at its stable equilibrium, Lyapunov's constraints for closed-loop stability are not enforced here. The decision variables are constrained to lie within the interval $[-40, 40]$, based on bounds determined through trial and error. A more

detailed analysis of the sensitivity of initial conditions with respect to the solver's cost can be found in subsection VII-A

$$\begin{aligned} \min_{A \in \mathbb{R}^{4 \times 4}, B \in \mathbb{R}^{4 \times 1}} \quad & \sum_{i=1}^{N-1} e_i^T Q e_i \\ \text{subject to:} \quad & e_i = x_{i+1}^{\text{true}} - (x_i + \Delta t_i (A x_i + B u_i)) \\ & -40 \leq a_{ij} \leq 40, \quad \forall i, j \in \{1, 2, 3, 4\} \\ & -40 \leq b_j \leq 40, \quad \forall j \in \{1, 2, 3, 4\} \end{aligned} \quad (12)$$

Since the optimization problem has a quadratic cost function with linear constraints, it is a Quadratic Program (QP).

A. Proving the Solution's Existence

In order to prove the existence of a solution, Weierstrass' Theorem is invoked. The theorem states that if a function f is continuous on a non-empty, compact set $S \subset \mathbb{R}^n$, then f attains both a global minimum and maximum on S . The cost function shown in Equation 5 is a sum of the quadratic expressions in A and B , which are polynomials and hence continuous.

As defined in subsection III-E, the feasible domain is constrained to lie within a closed and bounded set. Accordingly, there exists an optimal pair (A^*, B^*) such that

$$J(A^*, B^*) = \min_{A, B} J(A, B).$$

An example of a feasible solution is any matrix where $A \in \mathbb{R}^{4 \times 4}$ and $B \in \mathbb{R}^{4 \times 1}$, such that each entry in A and B lies within the interval $[-40, 40]$. A solution is defined as optimal if it satisfies the Second-Order Necessary Conditions (SONC), which include:

- The solution x^* satisfies all the constraints.
- The second derivative (Hessian) of the cost function at x^* is positive definite (PD). This holds for all feasible points, since the Hessian of the cost function Equation 5 is given by $\nabla^2 f(x) = 2Q$, and Q is defined to be positive definite.
- The gradient of the cost function, given by

$$\nabla J(x^*) = \sum_{i=1}^{N-1} 2Q(X_{\text{Estimated},i} - X_{\text{True},i}),$$

is zero at x^* .

B. casADi's Solver Specifics

To solve the optimization problem, the casADi [11] optimization framework is used in Python. While casADi supports a variety of solvers, IPOPT (Interior Point OPTimizer) is particularly well-suited for solving nonlinear programming (NLP) problems and is therefore selected in this work. This choice is especially important for the unstable equilibrium case, where the state-space identification involves nonlinear constraints, as discussed in subsection III-D. Consequently, IPOPT is used for solving both the stable and unstable formulations. The solver parameters used for IPOPT are listed in Table II, along with explanations for each parameter based on [11].

TABLE II: IPOPT Solver Tolerances

Parameter	Value
ipopt.tol	1×10^{-6}
ipopt.dual_inf_tol	1×10^{-6}
ipopt.constr_viol_tol	1×10^{-6}
ipopt.compl_inf_tol	1×10^{-6}
ipopt.acceptable_tol	1×10^{-5}
ipopt.acceptable_obj_change_tol	1×10^{-6}

- ipopt.tol: Sets the tolerance for convergence based on the change in the objective function.
- ipopt.dual_inf_tol: Defines the tolerance for the dual infeasibility condition, invalidating the solution if exceeded.
- ipopt.constr_viol_tol: Specifies the tolerance for allowable constraint violations before considering the solution infeasible.
- ipopt.compl_inf_tol: Sets the tolerance for complementary slackness, ensuring optimality for inequality constraints.
- ipopt.acceptable_tol: Defines the tolerance for an acceptable solution based on the objective function for stopping criteria.
- ipopt.acceptable_obj_change_tol: Sets the tolerance for an acceptable change in the objective function as a convergence criterion.

It is important to note that IPOPT searches for local minimum solutions when solving a NonLinear Program. Accordingly, the initial conditions and the constraints play a big role in forcing the solver to go towards the correct minimum. A visual explanation of this is provided in subsection VII-B.

VI. PROBLEM FORMULATION FOR UNSTABLE EQUILIBRIUM USING CASADI

The optimization problem formulation for unstable equilibrium is presented below. This case is more complex than the stable equilibrium due to the addition of Lyapunov's stability constraints on the closed-loop matrix. As discussed in subsection III-D, these constraints are enforced using Sylvester's criterion, which requires the determinants of the leading principal minors to alternate in sign to ensure the negative definiteness of the closed-loop matrix. The decision variables are constrained to lie within the interval $[-100, 100]$, based on bounds determined through trial and error. A more detailed analysis of the sensitivity of initial conditions with respect to the solver's cost can be found in subsection VII-B

$$\min_{A \in \mathbb{R}^{4 \times 4}, B \in \mathbb{R}^{4 \times 1}} \sum_{i=1}^{N-1} e_i^T Q e_i \quad (13)$$

subject to:

$$e_i = (x_{i+1}^{\text{true}} - (x_i + \Delta t_i(Ax_i + Bu_i)))^\top \quad \det[a_{11}] \leq \epsilon \quad (14)$$

$$\det \begin{bmatrix} \frac{a_{11}}{2} & \frac{a_{12}+a_{21}}{2} \\ \frac{a_{12}+a_{21}}{2} & a_{22} \end{bmatrix} \geq \epsilon \quad (15)$$

$$\det \begin{bmatrix} \frac{a_{11}}{2} & \frac{a_{12}+a_{21}}{2} & \frac{a_{13}+a_{31}}{2} \\ \frac{a_{12}+a_{21}}{2} & a_{22} & \frac{a_{23}+a_{32}}{2} \\ \frac{a_{13}+a_{31}}{2} & \frac{a_{23}+a_{32}}{2} & a_{33} \end{bmatrix} \leq \epsilon \quad (16)$$

$$\det \begin{bmatrix} \frac{a_{11}}{2} & \frac{a_{12}+a_{21}}{2} & \frac{a_{13}+a_{31}}{2} & \frac{a_{14}+a_{41}}{2} \\ \frac{a_{12}+a_{21}}{2} & a_{22} & \frac{a_{23}+a_{32}}{2} & \frac{a_{24}+a_{42}}{2} \\ \frac{a_{13}+a_{31}}{2} & \frac{a_{23}+a_{32}}{2} & a_{33} & \frac{a_{34}+a_{43}}{2} \\ \frac{a_{14}+a_{41}}{2} & \frac{a_{24}+a_{42}}{2} & \frac{a_{34}+a_{43}}{2} & a_{44} \end{bmatrix} \geq \epsilon \quad (17)$$

$$-100 \leq a_{ij} \leq 100, \quad \forall i, j \in \{1, 2, 3, 4\} \quad (18)$$

$$-100 \leq b_j \leq 100, \quad \forall j \in \{1, 2, 3, 4\} \quad (19)$$

Clearly, the constraints due to the Lyapunov's Stability Criteria make the problem a NLP (NonLinear Program). To solve the optimization problem, casADi's Interior Point Method (IPOPT) solver is used.

A. Proving the Solution's Existence

Similarly to the stable equilibrium problem subsection V-A, Weierstrass's Theorem can be invoked to prove that there exists an optimal pair (A^*, B^*) that minimizes the cost function. The cost function (J) still consists of polynomial expressions of the A and B matrices and the domain is a compact set. A feasible solution according to the given constraints:

$$(A \ B) = \left(\begin{bmatrix} -1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & -1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ 1 \\ 0 \end{bmatrix} \right)$$

The closed-loop eigenvalues for this system are $[-1, -1, -1, -1]$ which satisfies Lyapunov's constraints. The optimal solution should still satisfy the Second-Order Necessary Conditions (SONC) described in subsection V-A.

VII. RESULTS

A. Stable Equilibrium Optimization Results

In order to acquire data around the stable equilibrium point, the pendulum is positioned at the stable equilibrium (vertically downward position). A constant force of 0.5 N is applied to the cart and the four resulting state variables are recorded from the Gazebo simulator. The true state trajectories captured are shown in Figure VII.1 and Figure VII.2.

To initialize the solver, the following initial conditions are used: the matrix A is initialized as a 4×4 matrix with all entries set to 1, and the vector B is initialized as a 4×1 vector with all entries equal to 1. Despite being chosen randomly, the initial conditions don't really effect the solver as discussed later.

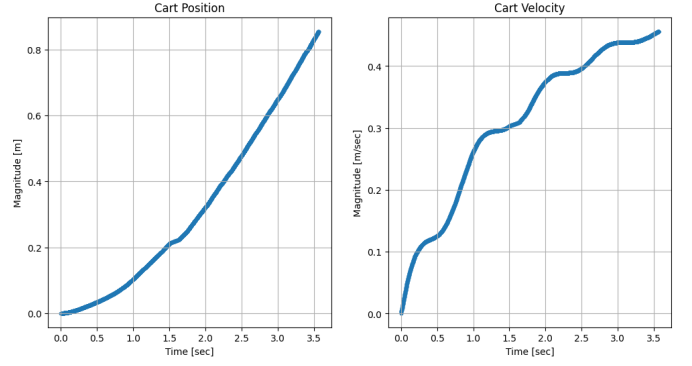


Fig. VII.1: Cart position and velocity true states for the stable equilibrium

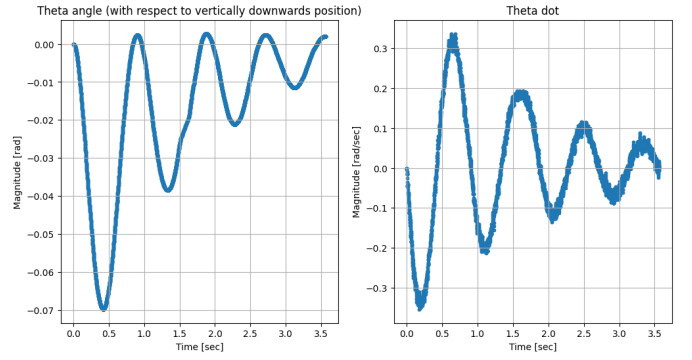


Fig. VII.2: Theta (θ) and Angular Velocity($\dot{\theta}$) true states obtained for stable equilibrium.

Solver Results

$$\text{Estimated } A = \begin{bmatrix} 0.028 & 0.018 & 0.029 & -0.14 \\ 0.069 & -0.15 & 4.10 & -40.0 \\ 0.42 & -0.015 & -0.92 & 3.85 \\ -0.020 & 0.48 & 0.019 & -0.84 \end{bmatrix}$$

$$\text{Estimated } B = \begin{bmatrix} 0.019 \\ -0.011 \\ 0.86 \\ -3.70 \end{bmatrix}$$

Cost: 0.0817

Solver status: Solve_Succeeded

Iteration count: 9

Time: 0.1sec

To verify the validity of the estimated dynamics, the poles of the A matrix are computed, as shown in Figure VII.3. As expected, one pair of complex conjugate poles lies in the left-half plane, representing the decaying oscillatory nature of the pendulum's motion.

Following are some observations about the optimized decision variables:

- The B matrix terms corresponding to θ and cart position have opposite signs. This holds true because the cart

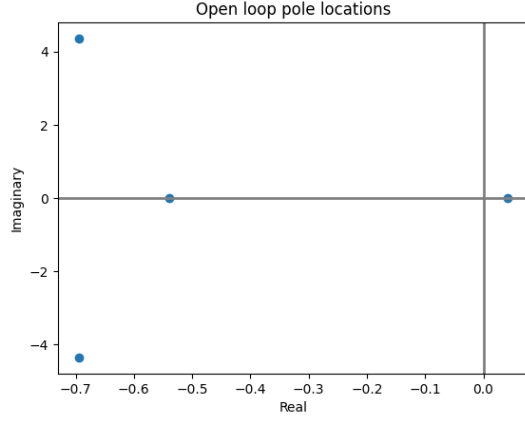


Fig. VII.3: Poles of the open-loop A matrix for the stable equilibrium

accelerates in the direction of the force applied, but the pendulum accelerates due to the opposite reaction of the force.

- The variable $A[3][4]$ is greater than 0, which makes sense. Anytime the pendulum swings to one extreme, it provides momentum to the entire system in the same direction, hence adding a boost to the cart's acceleration.
- The high correlation between $\dot{\theta}$ and $\ddot{\theta}$ could be a byproduct of the regression model, rather than a true underlying physical dependency.

Another validation criterion involves analyzing the free-swinging motion of the pendulum around the stable equilibrium. A small perturbation in this region should cause the pendulum to return to equilibrium after a few oscillations. To test this, an initial angular displacement of -0.14 radians is applied to the pendulum, and the open-loop dynamics are simulated over a 10-second period. A similar perturbation is applied in the Gazebo simulator for comparison. The results, shown in Figure VII.4, demonstrate the fidelity of the estimated model.

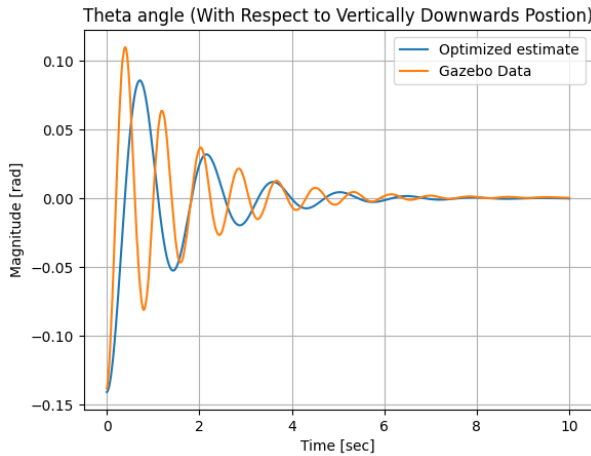


Fig. VII.4: Comparison of true and estimated free-swinging motion

To investigate the sensitivity of the optimization to initial

conditions and constraints, 20 simulations are run with varying parameters. The first experiment tests the sensitivity of the cost function to the initial conditions of the matrix A . A random number generator is used to scale the initial conditions for the A matrix, while keeping those of B fixed. Note, the baseline results are generated using matrices of ones for both A and B , so a scaling factor of 1 corresponds to this default case. The percent change is computed for both the cost and the scaling factor, and the results are shown in Figure VII.5.

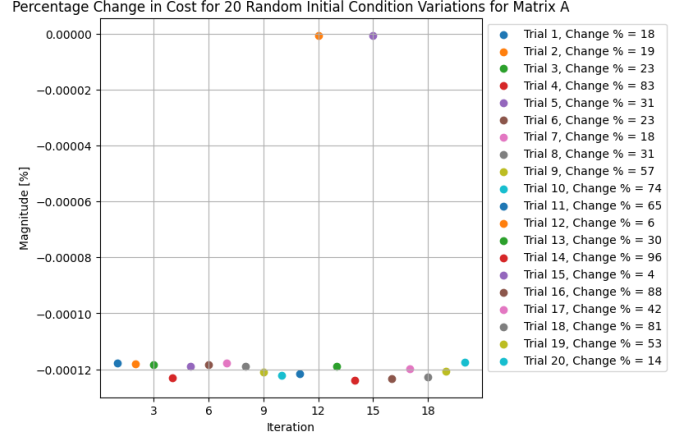


Fig. VII.5: Integrated response of optimized systems for 20 different initializations at the stable equilibrium.

The results indicate that changes in the initial conditions for the matrix A do not affect the cost meaningfully. In particular, the two cases with cost percentage changes near 0% correspond to initial condition changes of only 6% and 4%, respectively. The average cost percentage change is approximately -0.00012% , supporting the hypothesis that a global minimum exists and that the solver consistently finds it. A similar study is conducted for the initial conditions of the B matrix, with results shown in Figure VII.6.

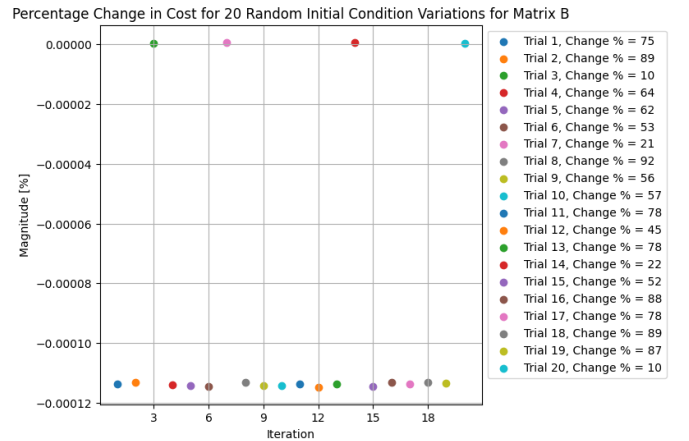


Fig. VII.6: Integrated response of optimized systems for 20 different initializations at the stable equilibrium.

As with matrix A , the percentage change in cost is negligible, providing additional evidence for the existence of a global minimum. Next, the initial conditions for both A and

B matrices are varied simultaneously and the estimated state-space models are validated using an initial perturbation of -0.14 rad. The results are shown in Figure VII.7.

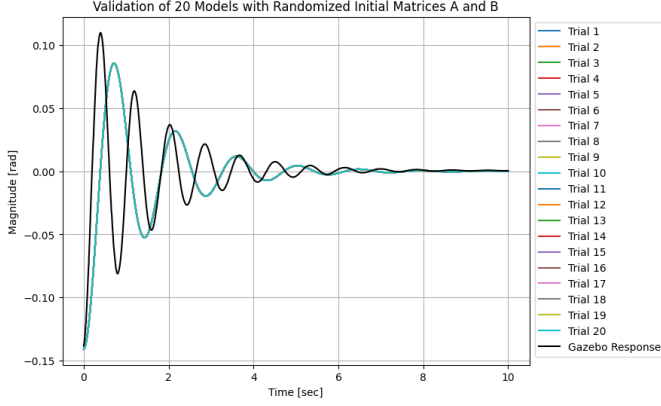


Fig. VII.7: Integrated response of optimized systems for 20 different initializations at the stable equilibrium.

As expected, all 20 responses are virtually identical since the cost percentage changes are negligible. A similar sensitivity analysis is conducted on the physical constraints imposed on the decision variables. The baseline constraints use bounds of -40 and 40 on all decision variables. A random number generator is used to scale these bounds, and cost percentage changes are measured for 20 runs. The results are shown in Figure VII.8. Unlike the study on initial conditions, variation

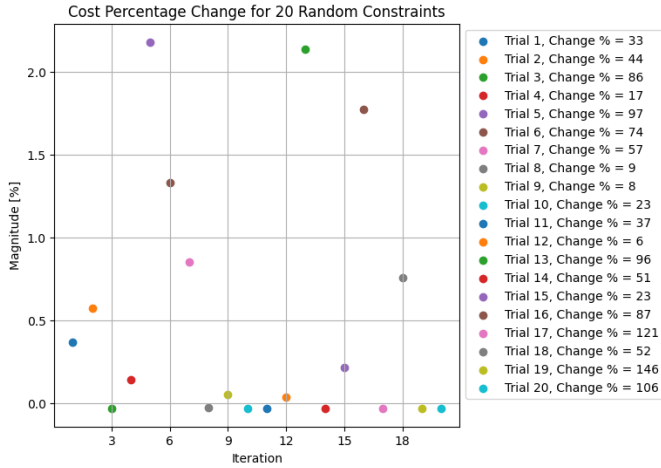


Fig. VII.8: Integrated response of optimized systems for 20 different initializations at the stable equilibrium.

in the constraints does result in noticeable changes in cost. While the differences are not large, they are significant enough to affect the optimization outcome. To visualize these effects, the estimated models from 20 similar trials are validated using the same -0.14 rad initial perturbation. The results are shown in Figure VII.9.

Clearly, as the constraint variation increases, the solver's ability to fit the data consistently is reduced. The resulting models become more sluggish, demonstrating the importance of well-defined constraint bounds for reliable optimization.

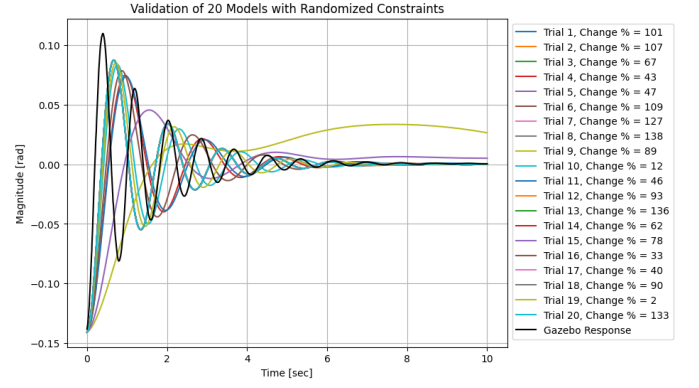


Fig. VII.9: Integrated response of optimized systems for 20 different initializations at the stable equilibrium.

B. Unstable Equilibrium Optimization Results

The true states of the stabilized inverted pendulum model are found in Figure VII.10, Figure VII.11, and Figure VII.12. As expected, the θ angle oscillates due to the oscillatory control action and then stabilizes. A video of this simulation can be found in [6].

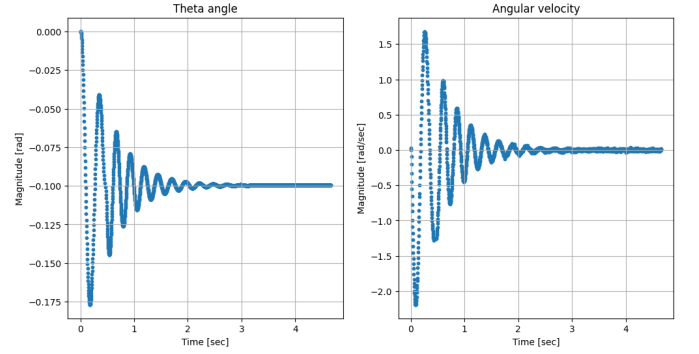


Fig. VII.10: Theta (θ) and Angular velocity ($\dot{\theta}$) true states for the unstable equilibrium.

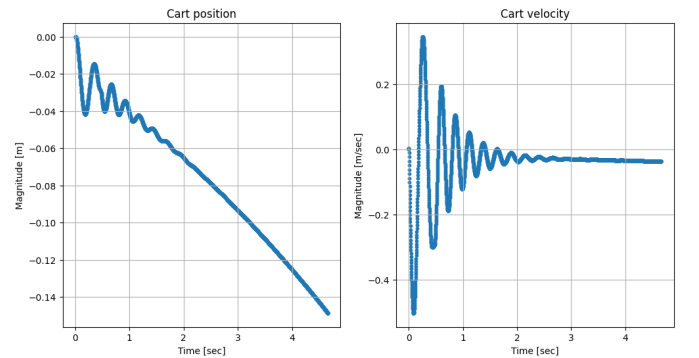


Fig. VII.11: Cart position and velocity true states for the unstable equilibrium.

To initialize the solver, the following initial conditions are used: the matrix A is initialized as a 4×4 matrix with all entries set to 0, and the vector B is initialized as a 4×1 vector with all entries equal to 0. These values are chosen

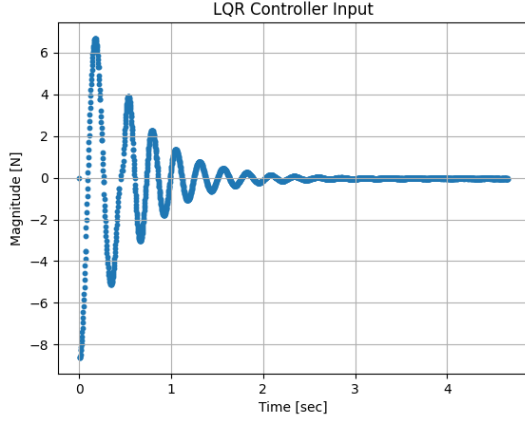


Fig. VII.12: The LQR control input for the unstable equilibrium.

through trial and error to ensure solver finds the suitable local minima. This will be further explained later.

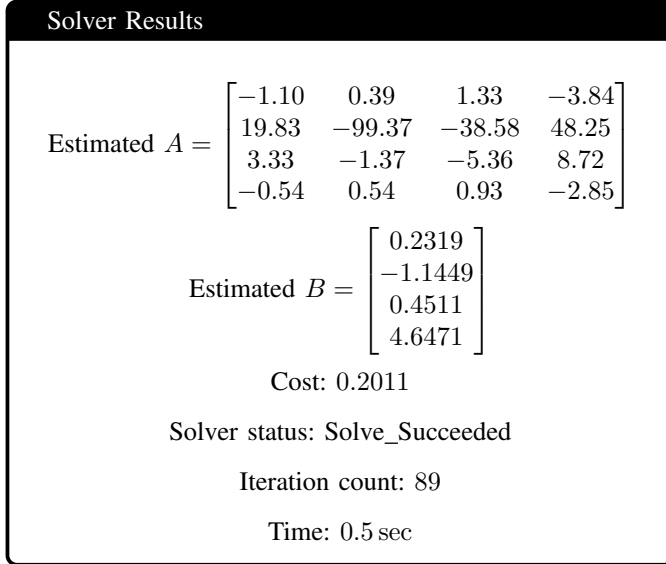


Fig. VII.13: The LQR control input obtained from Gazebo.

This hypothesis is validated by simulating the open-loop dynamics with an initial perturbation of -0.14 rad. As shown in Figure VII.14, the system response diverges toward the unstable equilibrium even in the absence of actuation.

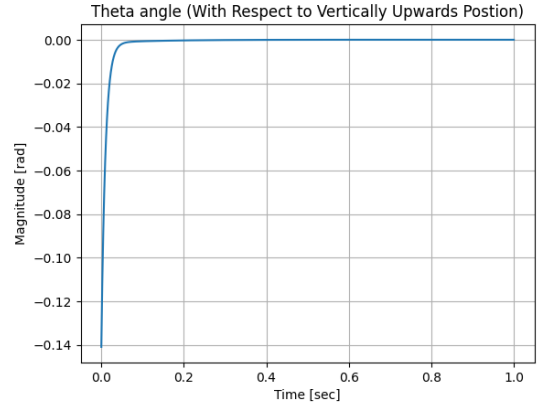


Fig. VII.14: The LQR control input obtained from Gazebo.

To verify the validity of the estimated dynamics, the open- and closed-loop poles of the A matrix are computed, as shown in Figure VII.13. As expected, the closed-loop poles of the system are stable—the real parts of the eigenvalues are negative—and they represent the decaying oscillatory nature of the pendulum's motion.

Although it might seem counterintuitive that the open-loop poles are all real (i.e., non-complex), implying the absence of oscillatory behavior, this is a direct result of linearizing the system around the unstable equilibrium. One way to interpret this is that linear systems are only valid for small perturbations and fail to accurately represent dynamics beyond that regime. Consequently, when linearizing around the unstable equilibrium, the resulting model cannot capture the oscillatory behavior that would be observed at the stable equilibrium. This can also be shown mathematically by linearizing the nonlinear inverted pendulum system at the unstable equilibrium and observing that the eigenvalues are real. A supporting example can be found in [12].

Next, a sensitivity analysis of the cost function with respect to the initial conditions on matrices A , B , and the constraints is performed using the same methodology as for the stable equilibrium case: a random scaling factor is applied to perturb the parameters one at a time over 10 trials. First, this analysis is conducted for the initial conditions on matrix A , and the results are shown in Figure VII.15.

The solver exhibits high sensitivity to the initial conditions of matrix A . Notably, even a small perturbation—such as a 2% change—can cause the solver to fail, as indicated by the 'x' markers. Furthermore, the cost function varies significantly across trials, suggesting a landscape with many local minima in which the solver can get trapped. A similar sensitivity analysis is conducted for initial conditions on matrix B , with results shown in Figure VII.16.

The solver exhibits a higher success rate in converging when varying the initial conditions on matrix B ; however, no clear pattern emerges. For instance, successful convergence is observed at both low and high percentage changes, whereas failures occur around 18% and 74%. This inconsistency suggests that the optimization landscape may have irregular local

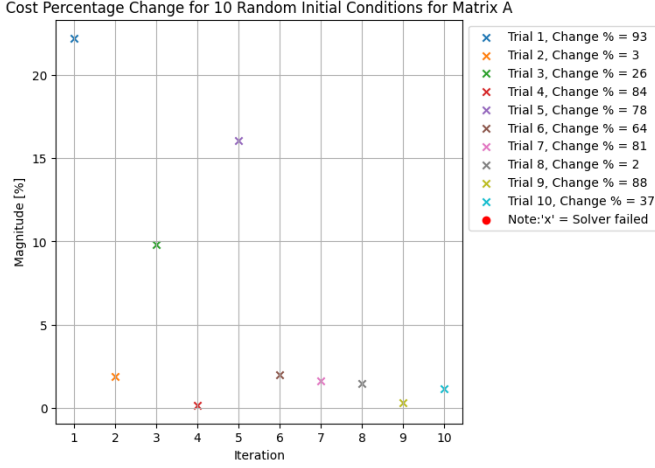


Fig. VII.15: Cost percentage change for 10 random initializations of matrix A at the unstable equilibrium. The 'x' marker indicates failed solver attempts.

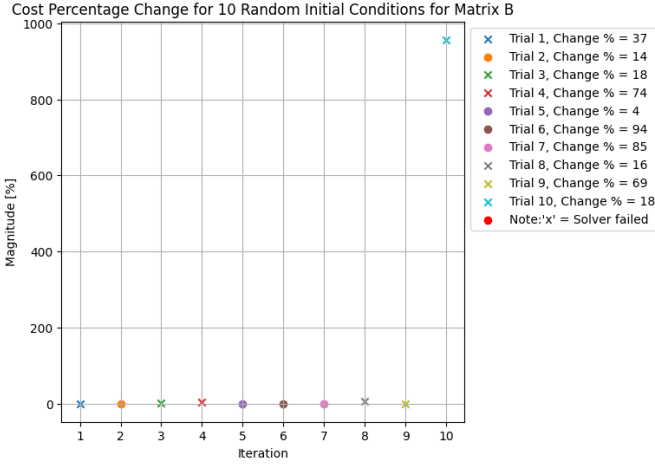


Fig. VII.16: The LQR control input obtained from Gazebo.

minima or sensitivity to certain regions of the parameter space. Next, a similar sensitivity analysis is conducted with respect to variations in the constraints on the feasible domain. The results are presented in Figure VII.17.

The cost is least sensitive to variations on the constraints. The solver finds a solution frequently, but this could be considered a consequence of the fixed starting conditions. Since the initial conditions are kept constant—as in the initial solve with zero matrices—the bounds do not significantly affect the cost. This observation reinforces the idea that the sensitivity of the solver is dominated more by the choice of initial conditions than by the feasible domain constraints.

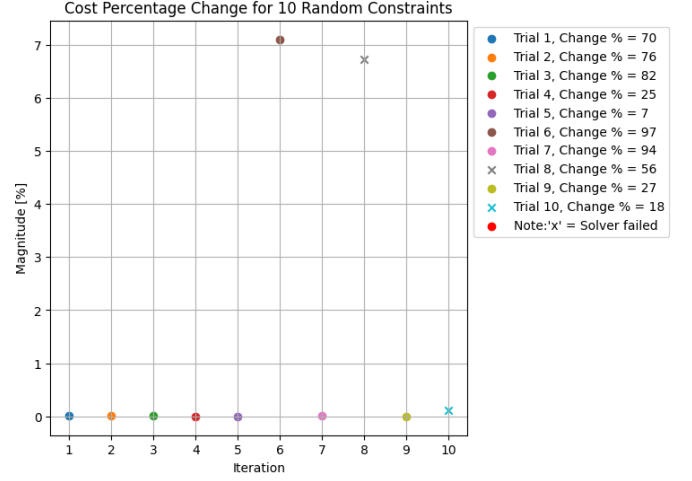


Fig. VII.17: The LQR control input obtained from Gazebo.

VIII. CONCLUSION

In this work, a linear system approximation of the nonlinear dynamics of an inverted pendulum about its stable equilibrium is successfully determined. The squared error at the stable equilibrium is 0.0817 %, indicating an excellent fit. Notably, it is observed that linear estimation of passive dynamics (i.e., without actuation), such as those near the stable equilibrium, is more likely to yield useful models.

In contrast, for the unstable equilibrium, although the squared error remains relatively low at 0.2011 %, the resulting linear model is less effective, as it fails to capture critical behaviors of the pendulum, most notably its oscillatory nature. The identified system matrices for both cases are presented below.

$$A_{\text{stable}} = \begin{bmatrix} 0.028 & 0.018 & 0.029 & -0.14 \\ 0.069 & -0.15 & 4.10 & -40.0 \\ 0.42 & -0.015 & -0.92 & 3.85 \\ -0.02 & 0.48 & 0.019 & -0.84 \end{bmatrix}$$

$$B_{\text{stable}} = \begin{bmatrix} 0.019 \\ -0.011 \\ 0.86 \\ -3.70 \end{bmatrix}$$

$$A_{\text{unstable}} = \begin{bmatrix} -1.10 & 0.39 & 1.33 & -3.84 \\ 19.83 & -99.37 & -38.58 & 48.25 \\ 3.33 & -1.37 & -5.36 & 8.72 \\ -0.54 & 0.54 & 0.93 & -2.85 \end{bmatrix}$$

$$B_{\text{unstable}} = \begin{bmatrix} 0.2319 \\ -1.1449 \\ 0.4511 \\ 4.6471 \end{bmatrix}$$

For future work, several avenues can be explored. Specifically, for system identification at the unstable equilibrium, additional constraints need to be incorporated to generate more useful models. Further validation techniques can also be employed, such as comparing system characteristics from the

closed-loop poles and Gazebo's response. Another potential direction is the design of LQR controllers based on the estimated systems at the unstable equilibrium.

REFERENCES

- [1] S. Sieniutycz, "Chapter 5 - systems design: Modeling, analysis, synthesis, and optimization," in *Complexity and Complex Thermo-Economic Systems*, S. Sieniutycz, Ed., Elsevier, 2020, pp. 85–115, ISBN: 978-0-12-818594-0. DOI: <https://doi.org/10.1016/B978-0-12-818594-0.00005-2>. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/B9780128185940000052>.
- [2] R. Johansson and L. Ljung, "Convex optimization in identification of stable nonlinear state space models," in *Proceedings of the 52nd IEEE Conference on Decision and Control*, IEEE, 2013, pp. 1574–1579.
- [3] R. Crevits and C. Croux, "Robust estimation of linear state space models," Faculty of Economics and Business, KU Leuven, Tech. Rep., 2020.
- [4] L. Vandenbergh, *Convex optimization techniques in system identification*, Accessed: 2025-05-02, 2009. [Online]. Available: <https://www.seas.ucla.edu/~vandenbe/publications/sysid-244.pdf>.
- [5] D. Liu, D. Sternad, and N. Hogan, "Stabilization of a cart inverted pendulum: Improving the intermittent feedback strategy to match the limits of human performance," *IEEE Transactions on Neural Systems and Rehabilitation Engineering*, vol. 13, no. 3, pp. 408–419, 2005. DOI: 10.1109/TNSRE.2005.848667.
- [6] K. Khushant, *Inverted pendulum*, https://github.com/khushant2001/Inverted_pendulum, Accessed: April 2025, 2024.
- [7] Open Source Robotics Foundation, *Robot Operating System 2 (ROS 2)*, <https://docs.ros.org/en/rolling/index.html>, Accessed: 2025-05-12, 2025.
- [8] Open Source Robotics Foundation, *Gazebo simulator*, <https://gazebo.org/>, Accessed: 2025-05-02, 2024.
- [9] Z. Gajić and M. T. J. Qureshi, *Lyapunov Matrix Equation in System Stability and Control* (Mathematics in Science and Engineering). San Diego: Academic Press, 1995, vol. 195, ISBN: 0-12-273370-3.
- [10] M. Lavrov, *Lecture 5: Sylvester's criterion and positive/negative definiteness*, <https://misha.fish/archive/docs/484-spring-2019/ch1lec5.pdf>, University of Illinois at Urbana-Champaign, 2019.
- [11] J. A. E. Andersson, J. Gillis, G. Horn, J. B. Rawlings, and M. Diehl, "CasADi – a software framework for nonlinear optimization and optimal control," *Mathematical Programming Computation*, vol. 11, no. 1, pp. 1–36, 2019. DOI: 10.1007/s12532-018-0139-4.
- [12] Control Tutorials for MATLAB and Simulink, *Inverted pendulum: System modeling*, Accessed: 2025-05-09, 2024. [Online]. Available: <https://ctms.engin.umich.edu/CTMS/index.php?example=InvertedPendulum§ion=SystemModeling>.
- [13] U. of Michigan, *Inverted pendulum: System modeling*, <https://ctms.engin.umich.edu/CTMS/index.php?example=InvertedPendulum§ion=SystemModeling>, Accessed: May 2, 2025.
- [14] MathWorks, *What are state-space models?* Accessed: 2025-05-09, 2025. [Online]. Available: <https://www.mathworks.com/help/ident/ug/what-are-state-space-models.html>.

IX. APPENDIX I. OPTIMIZATION SCRIPT FOR STABLE EQUILIBRIUM

Listing 1: CasADi Optimization Script

```

from casadi import *
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np

# Read the excel data!
true_data = pd.read_excel('Gazebo_sim_data.xlsx',
                          header = 0)

# Interpolating along the columns (axis = 0)
true_data = true_data.interpolate(axis = 0)

# Number of states = 4
n_states = 4
n_control = 1

# Defining the decision variables: A and B matrices
A = SX.sym('X', n_states, n_states)
B = SX.sym('U', n_states, n_control)

# Defining the PD matrix for the cost function!
Q = DM.eye(n_states)

# Initialize the cost function!
cost = 0

# Using a for loop to sum up the entire cost
function!
for i in range(len(true_data)-1):
    dt = true_data.iloc[i+1,1] - true_data.iloc[i,1]
    curr_state = DM([
        true_data.iloc[i,5], [true_data.iloc[i
        ,3]], [true_data.iloc[i,6]], [true_data.
        iloc[i,2]]
    ])
    true_next_state = DM([
        true_data.iloc[i+1,5], [true_data.iloc[i
        +1,3]], [true_data.iloc[i+1,6]], [
        true_data.iloc[i+1,2]]
    ])
    curr_input = DM([
        true_data.iloc[i,4]
    ])
    predicted_state = curr_state + dt*(A @
        curr_state + B @ curr_input)
    err = true_next_state - predicted_state
    cost += err.T @ Q @ err

# Define the constraints on the decision variables
lbw = DM(-40 * DM.ones(20, 1))
ubw = DM(40 * DM.ones(20, 1))

# Define the NLP problem in casADi.
opt_vars = vertcat(A.reshape((-1, 1)), B.reshape
    ((-1, 1)))
nlp_prob = {'x': opt_vars, 'f': cost}

opts = {
    "ipopt.print_level": 0,

```

```

    "print_time": False,
    "ipopt.tol": 1e-6,
    "ipopt.dual_inf_tol": 1e-6,
    "ipopt.constr_viol_tol": 1e-6,
    "ipopt.compl_inf_tol": 1e-6,
    "ipopt.acceptable_tol": 1e-5,
    "ipopt.acceptable_obj_change_tol": 1e-6,
}

solver = nlpsol('solver', 'ipopt', nlp_prob, opts)

# Provide the initial guesses!
A0 = 1*np.ones((4,4))
B0 = 1*np.ones((4,1))
x0 = np.concatenate([A0.flatten(), B0.flatten()])

# Run the Optimization Routine!
sol = solver(x0=x0, lbw=lbw, ubx=ubw)
# Get the eigenvalues
open_loop_eig, _ = np.linalg.eig(A_opt)

plt.figure()
plt.axhline(0, color='gray', linewidth=2)
plt.axvline(0, color='gray', linewidth=2)
plt.scatter(np.real(open_loop_eig), np.imag(
    open_loop_eig), label='Open_loop_poles')
plt.xlabel("Real")
plt.ylabel("Imaginary")
plt.title("Pole_locations")
plt.grid(True)
plt.legend()
plt.show()
print(closed_loop_eig)

# Sensitivity Analysis
plt.figure(figsize = (6,6))
for i in range(10):

    # Get the change using random number generator.
    # Can be used for changing A, B, and
    # constraints!
    change = 100*np.random.rand()
    print(change)
    A0 = np.zeros((4,4))
    B0 = np.zeros((4,1))
    x0 = np.concatenate([A0.flatten(), B0.flatten()
        ])

    lbw = DM(-change * DM.ones(20, 1))
    ubw = DM(change * DM.ones(20, 1))
    # Run the Optimization Routine!
    sol = solver(x0=x0, lbw =
        lower_bounds_constraints, ubg =
        upper_bounds_constraints, lbx = lbw, ubx =
        ubw)

    if solver.stats()['return_status'] != '
        Solved':
        plt.scatter(i+1, 100*(sol['f'].full()
            -0.20114865672564647)
            /0.20114865672564647, label = f'Trial_{i
            +1}', _Change_ = (int(abs((change-100)))
            ), marker = 'x')
    else:
        print(solver.stats()['return_status'])
        plt.scatter(i+1, 100*(sol['f'].full()
            -0.20114865672564647)
            /0.20114865672564647, label = f'Trial_{i
            +1}', _Change_ = (int(abs((change-100)))
            ), marker = 'x')

plt.scatter([], [], color='red', marker='o', label="
    Note: 'x' = Solver_failed")
plt.legend(loc='upper_left', bbox_to_anchor=(1, 1))
plt.grid()
plt.xlabel('Iteration')
plt.gca().xaxis.set_major_locator(MaxNLocator(
    integer=True))

```

```

plt.ylabel('Magnitude_[%]')
plt.title('Cost_Percentage_Change_for_10_Random_
    Constraints')

```

X. APPENDIX II. OPTIMIZATION SCRIPT FOR UNSTABLE EQUILIBRIUM

Listing 2: CasADi Optimization Script

```

from casadi import *
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np

# Read the excel data!
true_data = pd.read_excel('Gazebo_sim_data.xlsx',
    header = 0)

# Interpolating along the columns (axis = 0)
true_data = true_data.interpolate(axis = 0)

# Number of states = 4
n_states = 4
n_control = 1

# Defining the decision variables: A and B matrices
A = SX.sym('X', n_states, n_states)
B = SX.sym('U', n_states, n_control)

# Defining the PD matrix for the cost function!
Q = DM.eye(n_states)

# Initialize the cost function!
cost = 0

# Using a for loop to sum up the entire cost
# function!
for i in range(len(true_data)-1):
    dt = true_data.iloc[i+1,1] - true_data.iloc[i,1]
    curr_state = DM([
        [true_data.iloc[i,5]], [true_data.iloc[i
            ,3]], [true_data.iloc[i,6]], [true_data.
            iloc[i,2]]
    ])
    true_next_state = DM([
        [true_data.iloc[i+1,5]], [true_data.iloc[i
            +1,3]], [true_data.iloc[i+1,6]], [
            true_data.iloc[i+1,2]]
    ])
    curr_input = DM([
        [true_data.iloc[i,4]]
    ])
    predicted_state = curr_state + dt*(A @
        curr_state + B @ curr_input)
    err = true_next_state - predicted_state
    cost += err.T @ Q @ err

# Symmetric part of A_cl
M_sym = (A_cl + A_cl.T) / 2

# Sylvester's Criterion: Alternating signs for
# negative definite
minors = [
    det(M_sym[0:1, 0:1]), # > 0
    det(M_sym[0:2, 0:2]), # < 0
    det(M_sym[0:3, 0:3]), # > 0
    det(M_sym) # < 0
]

# Combine all constraints
constraints = vertcat(*minors)

# Bounds on the constraints
lower_bounds_constraints = vertcat(

```

```

    DM([-inf, 1e-6, -inf, 1e-6])
)

upper_bounds_constraints = vertcat(
    DM([-1e-6, inf, -1e-6, inf])
)

# Constraints on decision variables
lbx = DM([-100]*20)
ubx = DM([100]*20)

# Define the NLP problem in casADi.
opt_vars = vertcat(A.reshape((-1, 1)), B.reshape(
    ((-1, 1)))
nlp_prob = {'x': opt_vars, 'f': cost}

opts = {
    "ipopt.print_level": 0,
    "print_time": False,
    "ipopt.tol": 1e-6,
    "ipopt.dual_inf_tol": 1e-6,
    "ipopt.constr_viol_tol": 1e-6,
    "ipopt.compl_inf_tol": 1e-6,
    "ipopt.acceptable_tol": 1e-5,
    "ipopt.acceptable_obj_change_tol": 1e-6,
}

solver = nlpsol('solver', 'ipopt', nlp_prob, opts)

# Provide the initial guesses!
A0 = np.zeros((4,4))
B0 = np.zeros((4,1))
x0 = np.concatenate([A0.flatten(), B0.flatten()])

# Run the Optimization Routine!
sol = solver(x0=x0, lbg=lower_bounds_constraints,
    ubg=upper_bounds_constraints, lbx = lbx, ubx =
    ubx)

# Get the eigenvalues
open_loop_eig,_ = np.linalg.eig(A_opt)
closed_loop_eig,_ = np.linalg.eig(A_opt - B_opt@K)

plt.figure()
plt.axhline(0, color='gray', linewidth=2)
plt.axvline(0, color='gray', linewidth=2)
plt.scatter(np.real(open_loop_eig), np.imag(
    open_loop_eig), label='Open_loop_poles')
plt.scatter(np.real(closed_loop_eig), np.imag(
    closed_loop_eig), label='Closed_loop_poles')
plt.xlabel("Real")
plt.ylabel("Imaginary")
plt.title("Pole_locations")
plt.grid(True)
plt.legend()
plt.show()
print(closed_loop_eig)

# Sensitivity Analysis
plt.figure(figsize = (6,6))
for i in range(10):

    # Get the change using random number generator.
    # Can be used for changing A, B, and
    # constraints!
    change = 100*np.random.rand()
    print(change)
    A0 = np.zeros((4,4))
    B0 = np.zeros((4,1))
    x0 = np.concatenate([A0.flatten(), B0.flatten()
        ])

    lbw = DM(-change * DM.ones(20, 1))
    ubw = DM(change * DM.ones(20, 1))
    # Run the Optimization Routine!
    sol = solver(x0=x0, lbg =
        lower_bounds_constraints, ubg =
        upper_bounds_constraints, lbx = lbw, ubx =
        ubw)

    if solver.stats()['return_status'] != '
        Solved_Succeeded':
        plt.scatter(i+1,100*(sol['f'].full()
            -0.20114865672564647)
            /0.20114865672564647, label = f'Trial_{i
            +1},_Change_%_={int(abs((change-100)))}
            ', marker = 'x')
    else:
        print(solver.stats()['return_status'])
        plt.scatter(i+1,100*(sol['f'].full()
            -0.20114865672564647)
            /0.20114865672564647, label = f'Trial_{i
            +1},_Change_%_={int(abs((change-100)))}
            ')
    plt.scatter([], [], color='red', marker='o', label="
        Note:'x'=_Solver_failed")
    plt.legend(loc='upper_left', bbox_to_anchor=(1, 1))
    plt.grid()
    plt.xlabel('Iteration')
    plt.gca().xaxis.set_major_locator(MaxNLocator(
        integer=True))
    plt.ylabel('Magnitude_%')
    plt.title('Cost_Percentage_Change_for_10_Random_
        Constraints')

```